ICT305 Topics in IT

Topic 2
Deep Learning

CIHE Australia

Deep Learning

- Artificial Neurons
- Feedforward Neural Networks (FFNN)
- Deep Neural Network (DNN)
- Activation Functions
- Training a DNN
- Overfitting and How to avoid it

Deep Learning

Artificial intelligence

Any technique that enables machines to mimic human intelligence

Machine learning

Ability to learn without being explicitly programmed using past observations

Artificial neural networks

Extract patterns using neural networks

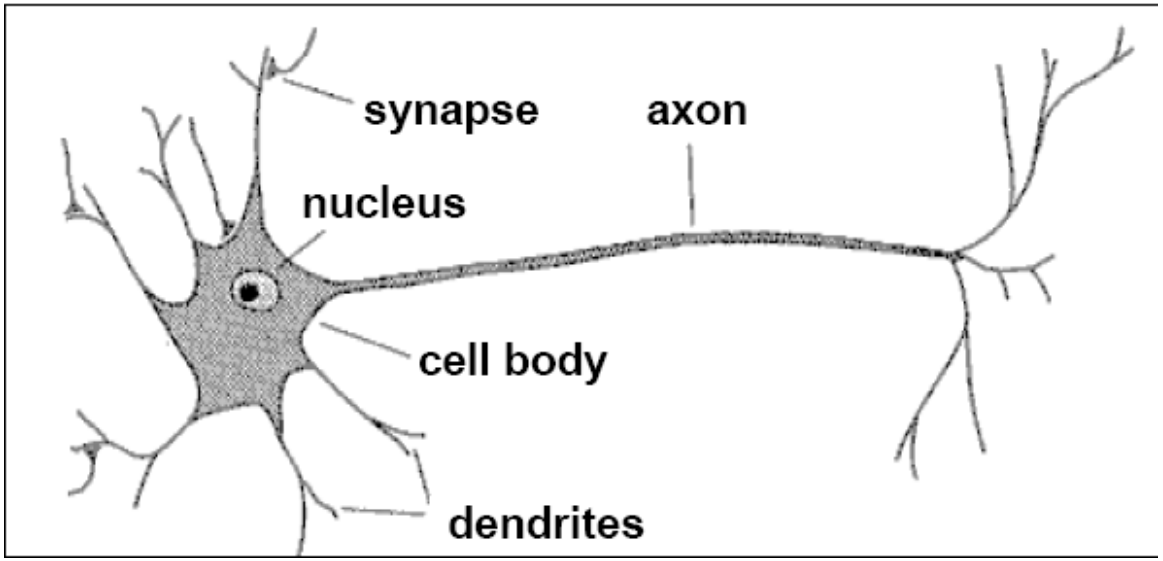
Deep learning

Modern revolution of neural networks

What is Artificial Neuron Networks?

- **An Artificial neural network** can be defined as a model of reasoning based on the human brain.
- **The brain** consists of a densely interconnected set of nerve cells (information processing units) called neurons.
  - Human brain has 10 billion neurons and 60 trillion connections.
- **ANN's** are a type of artificial intelligence that attempts to imitate the way a human brain works.
- The approach is beginning to prove useful in certain areas that involve recognizing complex patterns,
  - such as voice recognition and image recognition.

Biological Neuron

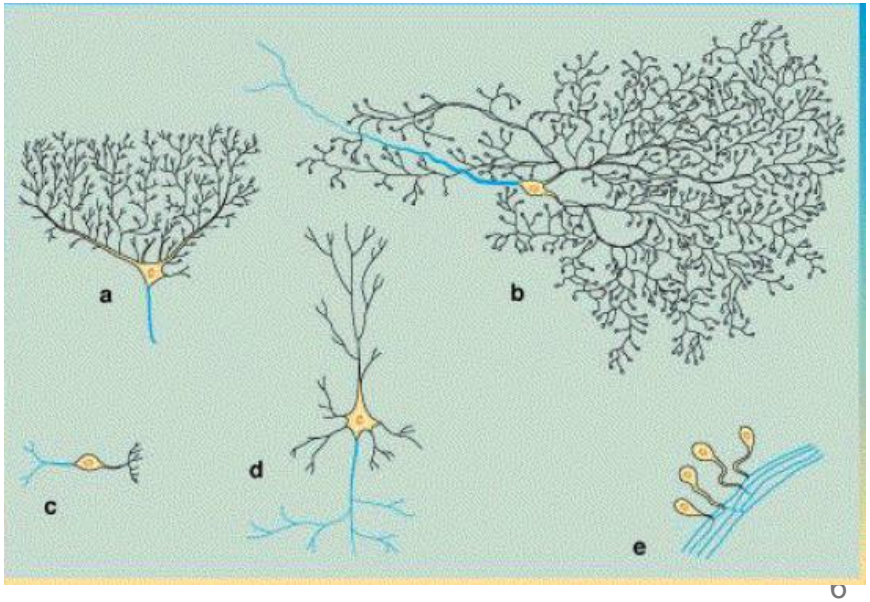


- A neuron has **a cell body**, a branching **input** structure, and a branching **output** structure (the axon).
- **Axons** connect to dendrites via synapses.
- **Electro-chemical signals** are propagated from the dendrites input, through the cell body, and down the axon to other neurons.

Structure of Neuron

- A neuron only fires if its input signal exceeds a certain amount (the threshold) in a short time period.
- Synapses vary in strength
  - Good connections allowing a large signal
  - Slight connections allow only a weak signal.
  - Synapses can be either excitatory or inhibitory.

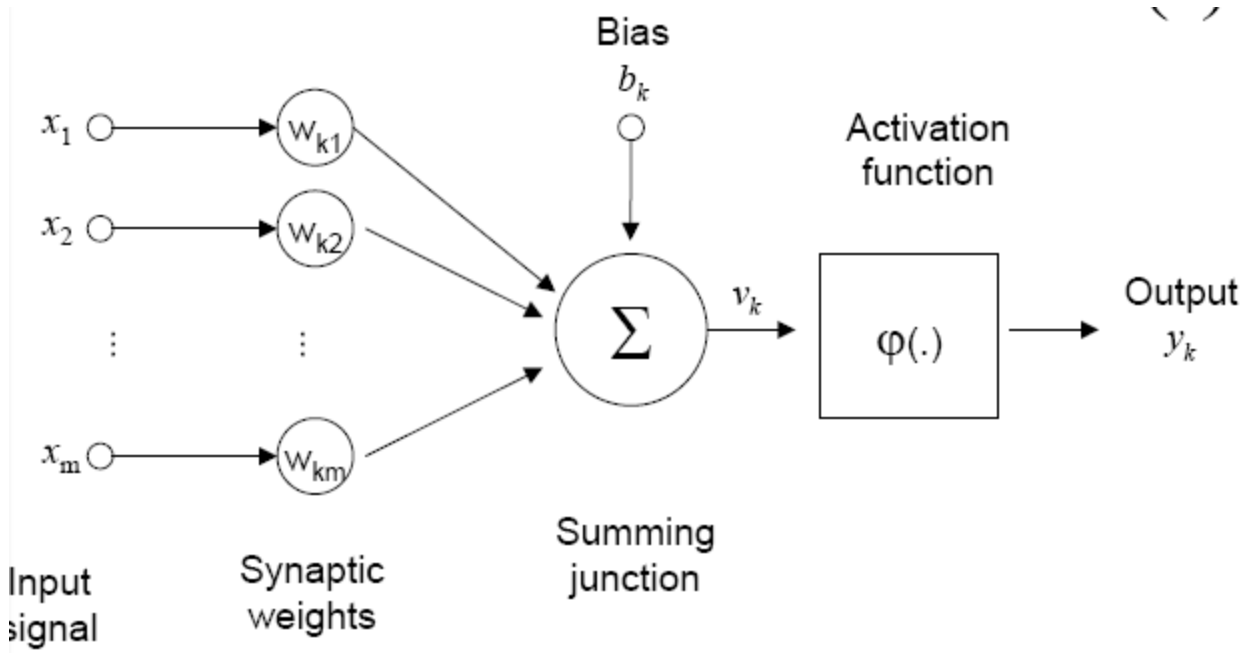
Neuron comes in many shapes and sizes



Models of Neuron

- **Neuron** is an information processing unit
- **A set of synapses** or connecting links
  - characterized by weight or strength
- **An adder**
  - summing the input signals weighted by synapses
  - a linear combiner
- **An activation function**
  - also called squashing function
    - -squash (limits) the output to some finite values

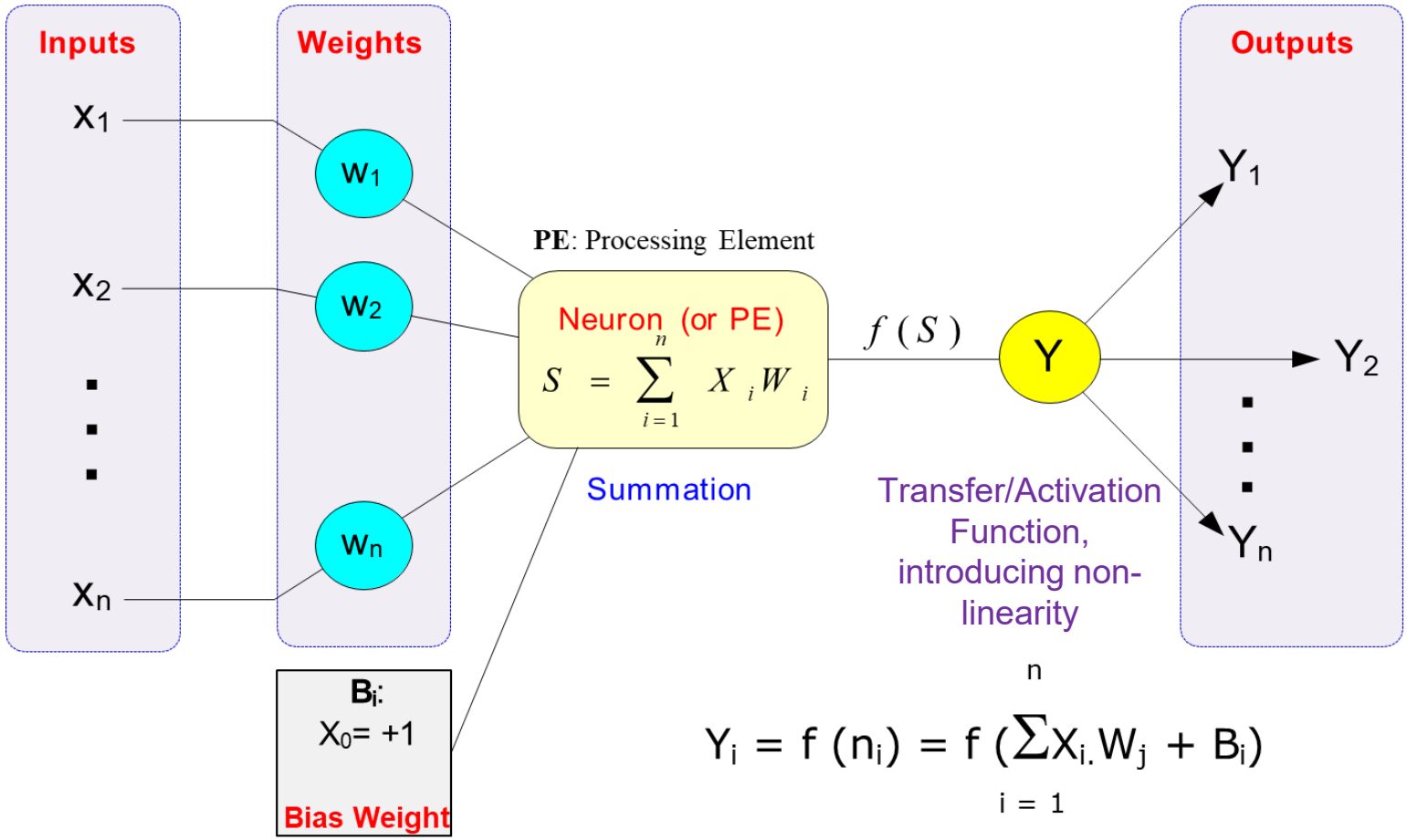
Model of Neuron (Perceptron)



$$v_k = \sum_{j=1}^{m} w_{kj} x_j + b_k \qquad y_k = \varphi(v_k)$$

- A perceptron is a single artificial neuron.
- It receives one or more inputs, performs a computation on those inputs, and produces an output.

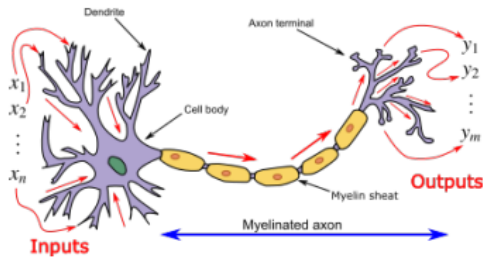
Model of Neuron (Perceptron)

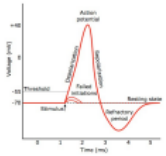


$$Y_i = f(n_i) = f\left(\sum_{i=1}^{n} X_i . W_j + B_i\right)$$

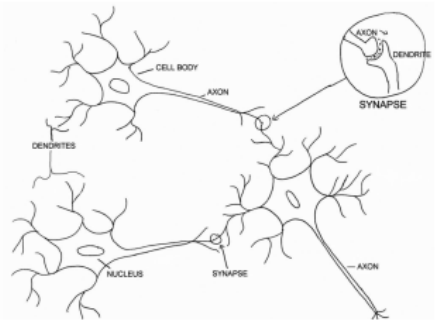
Analogy

Biological Neuron

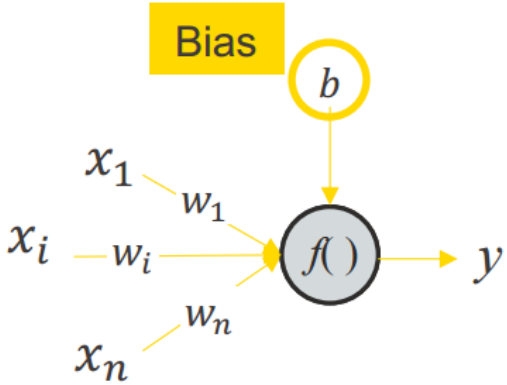




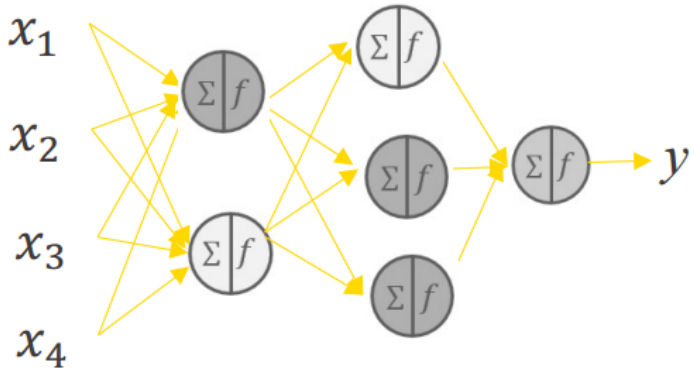
Biological Neural Networks



Artificial Neuron (**Perceptron**)



Artificial Neural Networks (**Multilayer Perceptron, MLP**)



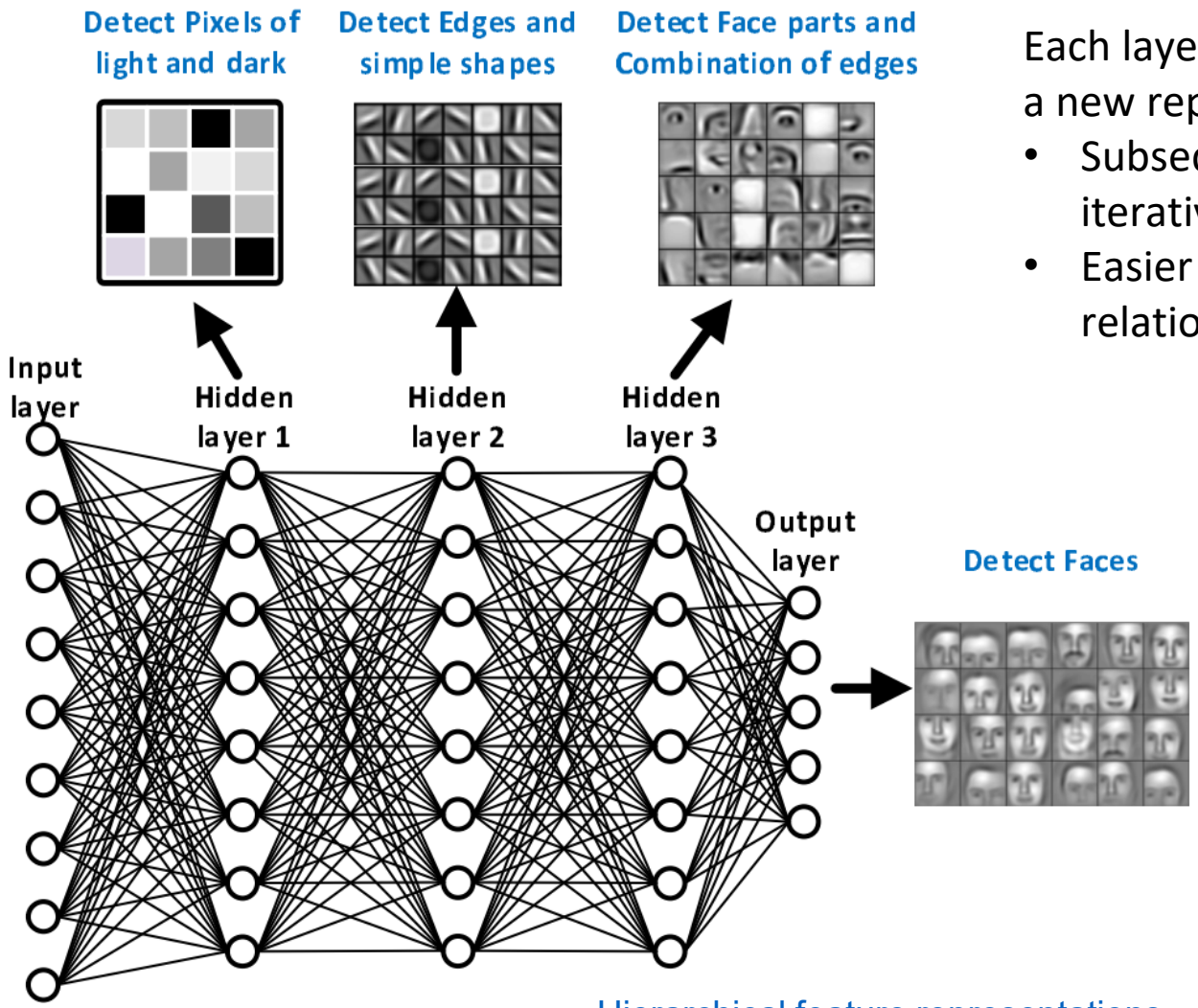
Requirements to Build an ANN

- How many neurons are to be used?
- How the neurons are to be connected to form a network.
- Which learning algorithm to use?
- How to train the neural network?
  - ***Training***: Initialize the weights of the network and update the weights from a set of training examples

Feedforward Neural Networks (FFNN)

- A network of artificial neurons with multiple layers
  - an input layer
  - one or more hidden layers
  - an output layer
  - Connections exist only between neurons from one layer to the next (feedforward)
  - MLP is a special case of FFNN, with non-linear activation functions.

Why layers?



Each layer acts as a filter and learns a new representation of the data

- Subsequent layers can learn iterative refinements
- Easier that learning a complex relationship in one go

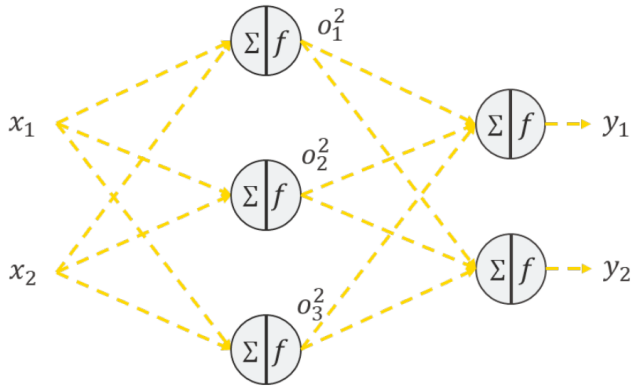
Hierarchical feature representations

Too Deep?

- **Vanishing and Exploding Gradients:**
  - **Vanishing Gradients**: In very deep networks, gradients of the loss function with respect to the weights can become very small as they are propagated back through the layers. This makes it difficult for the network to learn, as the updates to the weights become negligible.
  - **Exploding Gradients**: Conversely, gradients can also become excessively large, causing unstable updates and potentially leading to numerical overflow.
- **Overfitting:**
  - Deep networks have a large number of parameters, which increases their capacity to fit the training data.
  - This can lead to overfitting, where the model learns the noise and details of the training data rather than the underlying pattern, resulting in poor generalization to new data.
- **Increased Computational Cost**

FFNN: Example of Architecture / Topology

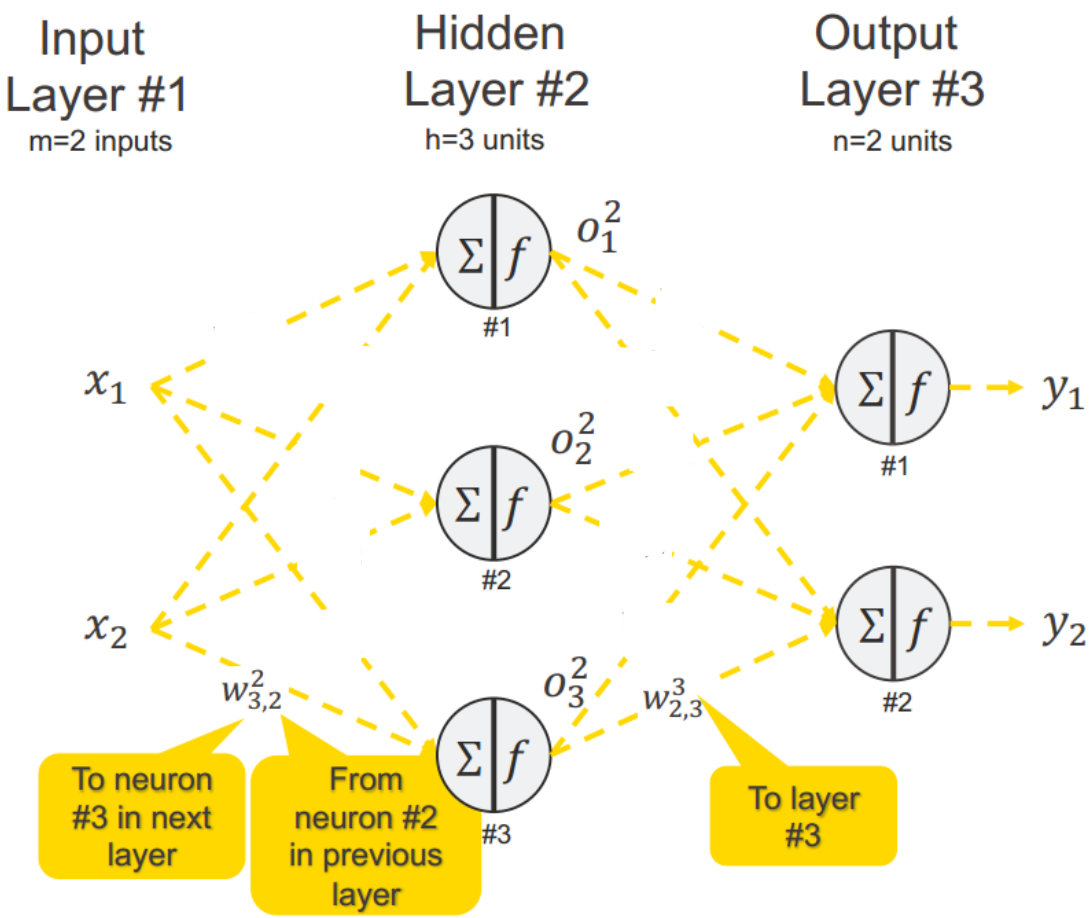
- Let's see an example of an FFNN
  - 1 input layer with m=2 inputs
  - 1 hidden layer with h=3 hidden neurons
  - 1 output layer with n=2 output neurons



fully connected feed forward neural network

  - **Fully-connected**: Each neuron in one layer is connected to all neurons in the next layers
  - **Dense layer**: fully connected to the previous layer

Feed-Forward Neural Networks (FFNN)



Forward pass:

$$o_j^2 = f\left(\sum_{i=1}^{n} w_{j,i}^2 x_i\right)$$

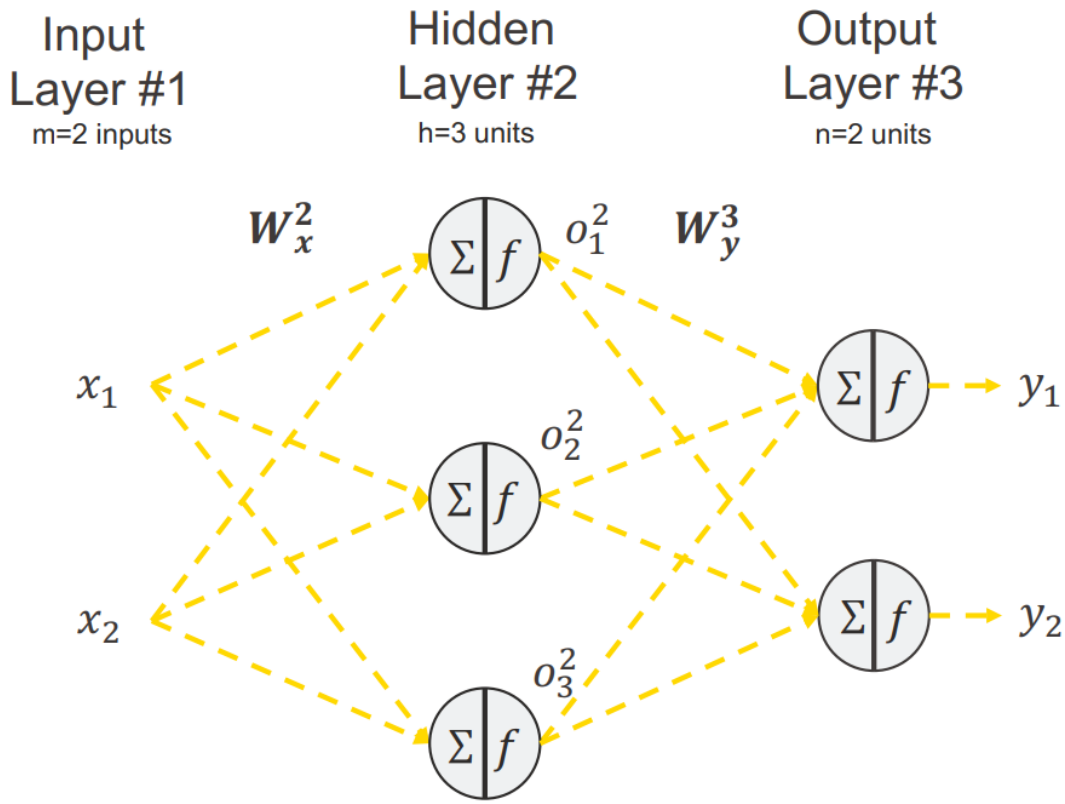
$$y_k = f\left(\sum_{j=1}^{h} w_{k,j}^3 o_j^2\right)$$

$$y_k = f\left(\sum_{j=1}^{h} w_{k,j}^3 f\left(\sum_{i=1}^{n} w_{j,i}^2 x_i\right)\right)$$

https://www.knime.com/sites/default/files/2021-08/l4-dl-slides.pdf

Feed-Forward Neural Networks (FFNN)

- Same with Matrix Notations



Forward pass:

$$\boldsymbol{o} = f(W_x^2 \boldsymbol{x})$$

$$y = f(W_y^3 \boldsymbol{o})$$

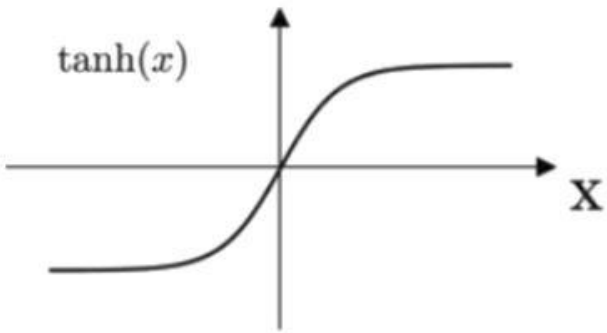
$$y = f(W_y^3 f(W_x^2 \boldsymbol{x}))$$

$f(\ )$ = activation function

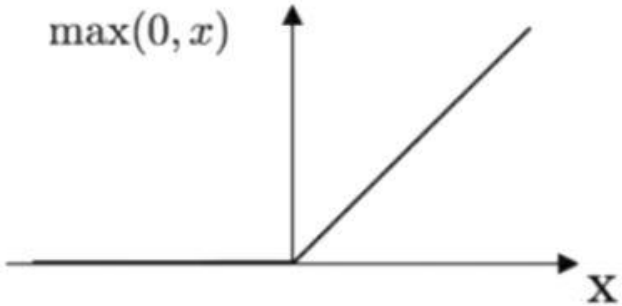
https://www.knime.com/sites/default/files/2021-08/l4-dl-slides.pdf

Frequently Used Activation Functions

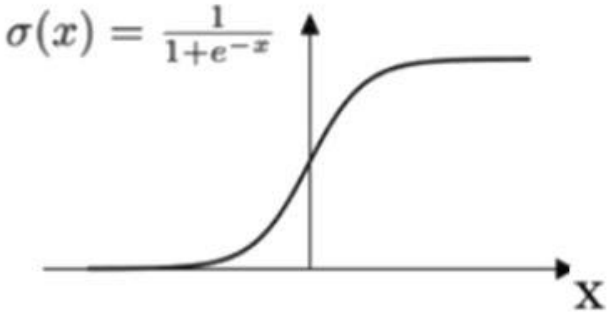
Tanh



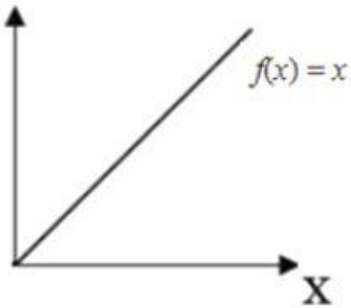
ReLU



Sigmoid



Linear



Which Activation Functions? Which Loss Functions?

✓ Recommended
Δ Can be used

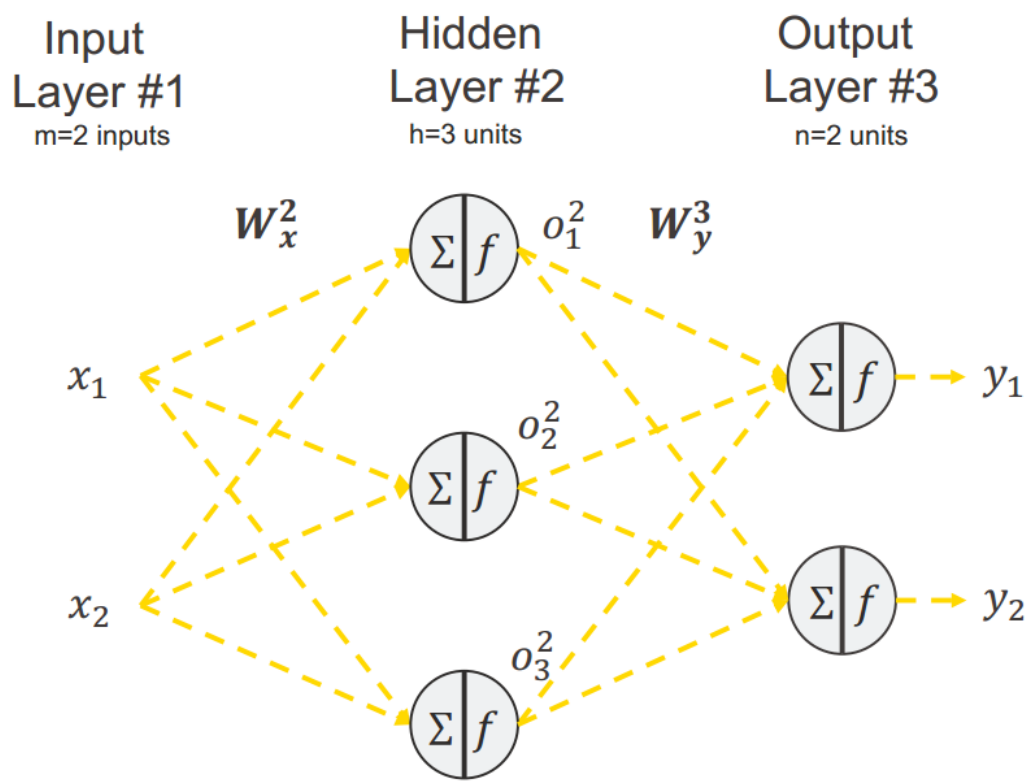
	Problems	Activation Functions								Loss Functions					
		Hidden Layers			Output Layer										
		Sigmoid	Tanh	ReLU	Sigmoid	Tanh	Linear	ReLU	Softmax	Binary CE	Hinge	Categorical CE	MSE	MSLE	MAE
Classification	Binary classification (0 vs 1)	✓	✓	✓	✓					✓					
	Binary classification (-1 vs 1)	✓	✓	✓		✓					✓				
	Multi-class classification	✓	✓	✓					✓			✓			
Regression	Regression	✓	✓	✓	Δ	Δ	✓	Δ					✓		
	Regression (wide range)	✓	✓	✓			✓							✓	
	Regression (possible outliers)	✓	✓	✓			✓								✓

Training a Feed Forward Neural Network

- **Teach** neuron(s) a desired input-output behavior
  - **Show** examples from the training set repeatedly
  - Networks adjusts parameters to fit underlying function:
    - Topology,
    - Weights,
    - Internal functional parameters.

Loss Function & Cost Function

Loss function quantify how well or poorly a model is performing by calculating the difference between predicted values and actual values.



$$\varepsilon = \frac{1}{2}\sum_{x \in T}\sum_{k=1}^{n}(y_k - t_k)^2$$

Number of output units

On the whole Training set

Network output value k

Target value k

Forward pass:

$$\boldsymbol{o} = f(W_x^2 \boldsymbol{x})$$

$$y = f(W_y^3 \boldsymbol{o})$$

Learning Rule from Gradient Descent

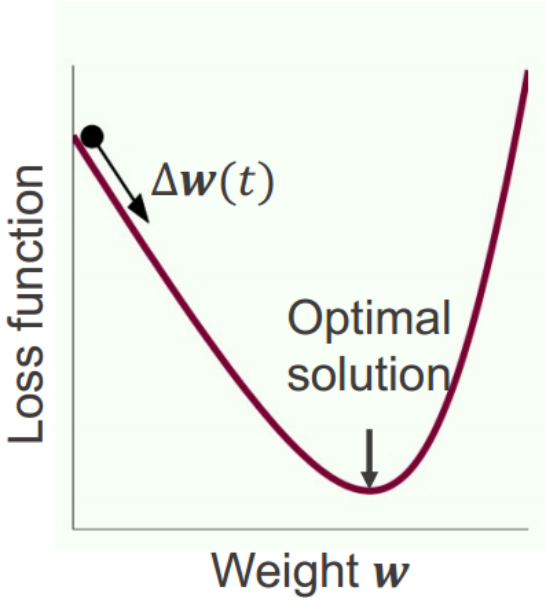
- Adjust the weight for the next step by the adjustment term $\Delta \boldsymbol{w}(t)$.

$$\boxed{\boldsymbol{w}(t+1)} = \boxed{\boldsymbol{w}(t)} + \boxed{\Delta \boldsymbol{w}(t)}$$

Updated weight in next step $t+1$

Weight in current step t

Weight adjustment based on gradient



https://www.knime.com/sites/default/files/2021-08/l4-dl-slides.pdf

Learning Rule from Gradient Descent

$\Delta \boldsymbol{w}$ has two parts:

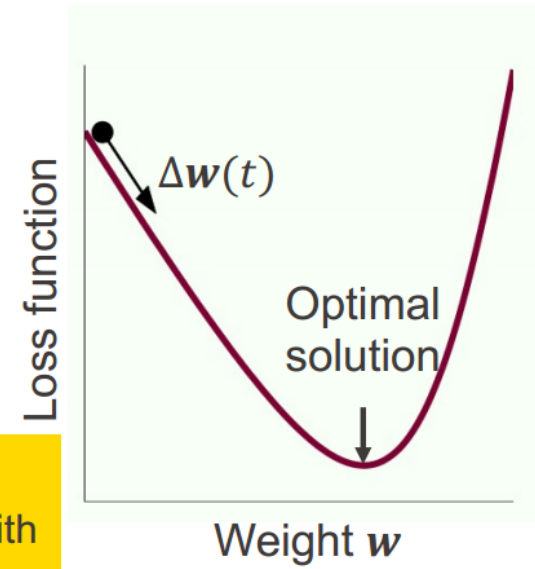
$$\Delta \boldsymbol{w}(t) = -\boxed{\eta}\boxed{\frac{\partial \varepsilon}{\partial \boldsymbol{w}}}$$

Need to calculate as many partial derivatives as the number of all weights!

Learning rate $\eta > 0$:
Dictates how quickly to approach the solution

Slope of
Loss function

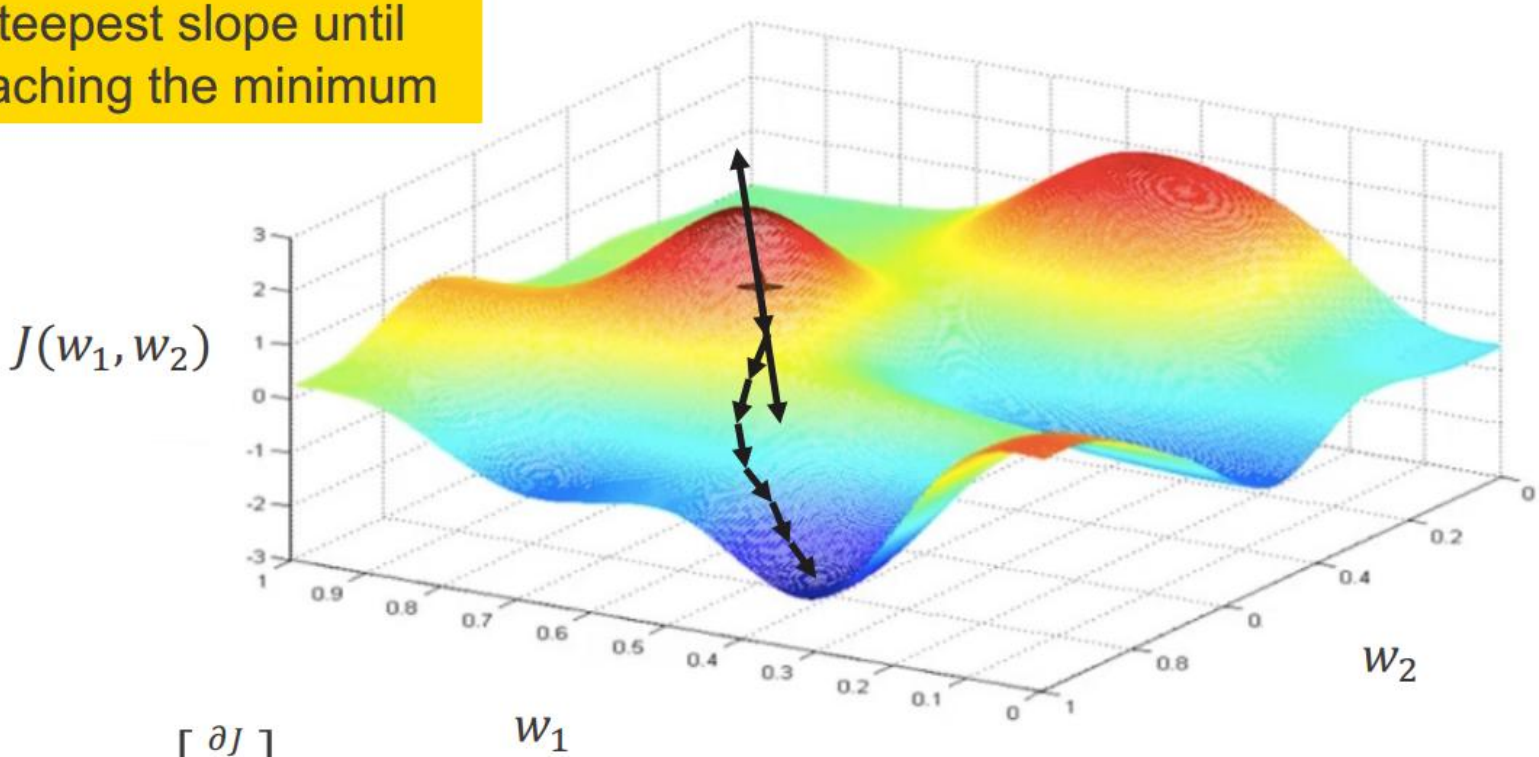
Gradient:
Partial derivatives with respect to weight $\boldsymbol{w}$



https://www.knime.com/sites/default/files/2021-08/l4-dl-slides.pdf

Idea Behind Gradient Descent

Rolling down the steepest slope until reaching the minimum



$$\nabla_W J(x, W) = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \frac{\partial J}{\partial w_2} \end{bmatrix}$$

https://www.knime.com/sites/default/files/2021-08/l4-dl-slides.pdf

Back-propagation

- Gradients can be determined – one layer at a time – by the chain rule.
- To determine the gradient at a particular layer, you only need gradients from the subsequent layers.

Why This Works

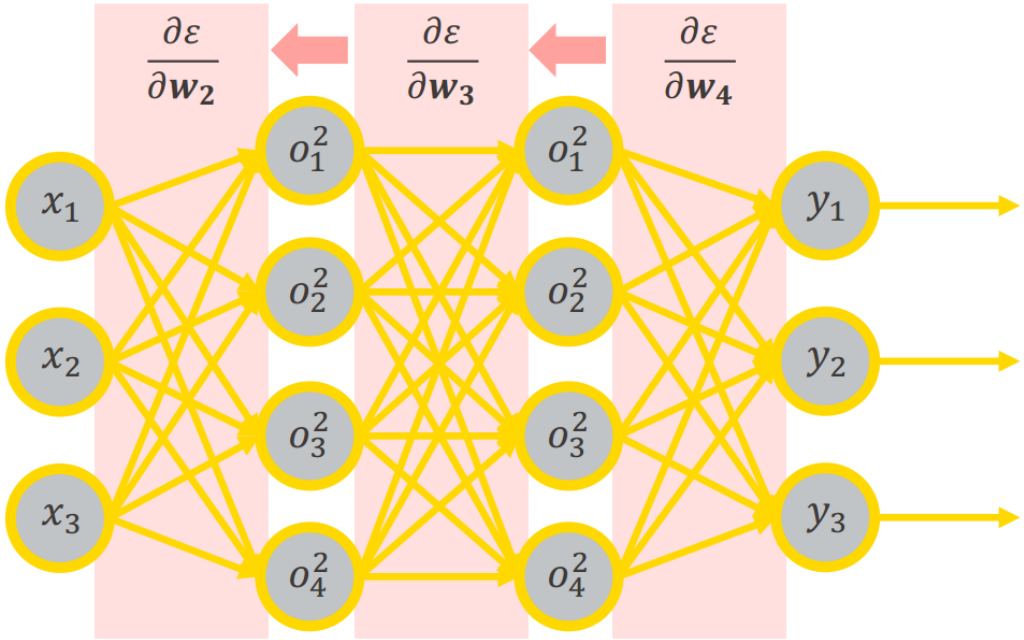
The key insight is the chain rule decomposition:

$$\frac{\partial \text{Cost}}{\partial z^{(l)}} = \frac{\partial \text{Cost}}{\partial z^{(l+1)}} \cdot \frac{\partial z^{(l+1)}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}}$$

Where:

- $\frac{\partial z^{(l+1)}}{\partial a^{(l)}} = W^{(l+1)}$ (since $z^{(l+1)} = W^{(l+1)} a^{(l)} + b^{(l+1)}$)
- $\frac{\partial a^{(l)}}{\partial z^{(l)}} = f'(z^{(l)})$ (activation derivative)

Symbol	Meaning
l	Current layer
$l+1$	The next layer (closer to output)
$z^{(l)}$	Weighted sum at layer l (before activation)
$a^{(l)}$	Activation output at layer l (after activation function)
$W^{(l+1)}$	Weights connecting layer l to $l+1$



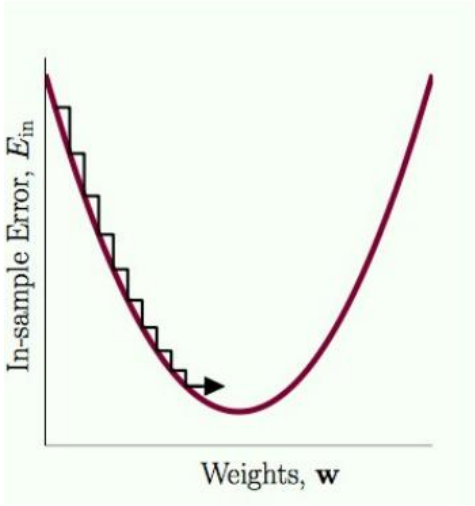

```
Forward Pass →→→→→→→→→→→→→→→→→→→→→→→
Input → Layer 1 → Layer 2 → Layer 3 → Output → Compute Cost
                                                          ↓
Backward Pass ←←←←←←←←←←←←←←←←←←←←←←←←←←←←←←
Update w2 ← Update w3 ← Update w4 ← ∂ε/∂w flows back
```

Learning Rate

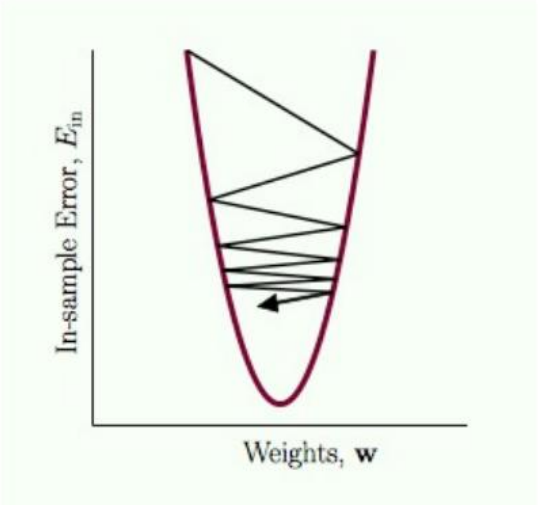
- Network weights are updated by back propagation
- How much weight adjustment?

  → controlled by the learning rate η
- Optimizers → various variants of stochastic gradient descent (SGD)

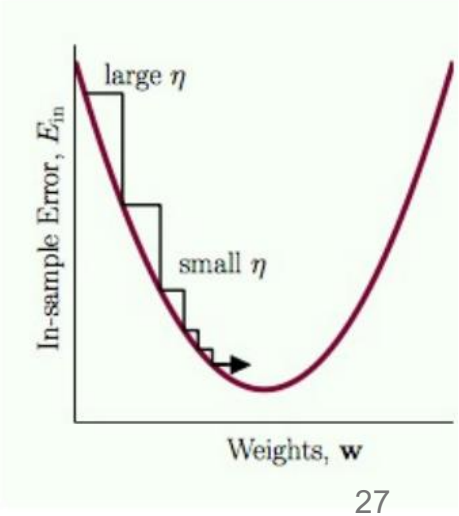
η too small



η too large

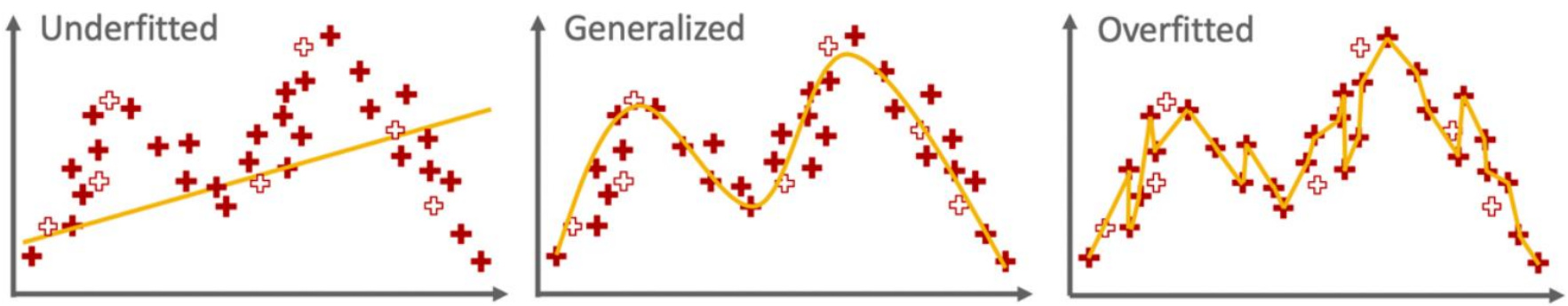


η just right



Overfitting and How to Avoid it

- **Overfitting**
  - Large neural network with many parameters trained on small dataset
  - Able to fit the training set very well, but unable to generalize to other data



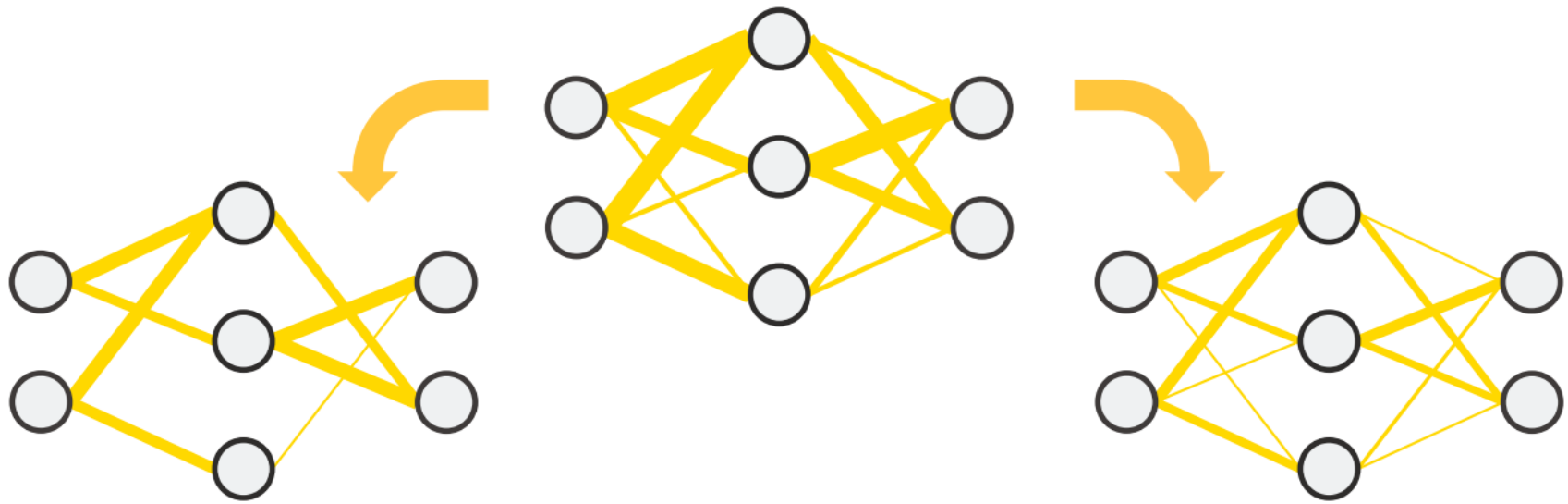
Avoiding Overfitting – Norm Regularization

- **One sign of overfitting** – high values of weights ***W***
- ***Solution***: penalty term in loss function to avoid large weights

$$\tilde{E}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = E(\boldsymbol{y}, \hat{\boldsymbol{y}}) + \boxed{\lambda \Omega(\boldsymbol{W})}$$

  - Parameter λ: controls the penalty effect – strong regularization with larger λ

Avoiding Overfitting – Norm Regularization



<u>L1 norm</u> – a.k.a., Laplace, LASSO

$$\Omega(\boldsymbol{W}) = L_1(\boldsymbol{W}) = \sum_{i=1}^{n} |\boldsymbol{W}_i|$$

- Can reduce some weights to zero → simplifying the network
- Robust to outliers

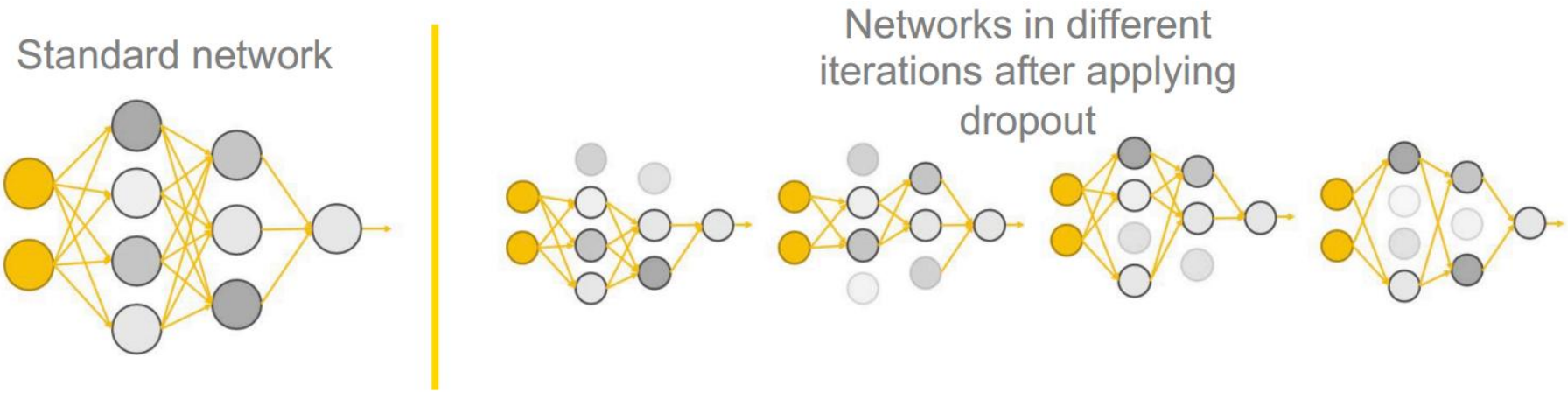
<u>L2 norm</u> – a.k.a., Gauss, Ridge

$$\Omega(\boldsymbol{W}) = L_2(\boldsymbol{W}) = \sum_{i=1}^{n} \boldsymbol{W}_i^2$$

- All connections are included
- Better prediction
- Sensitive to outliers

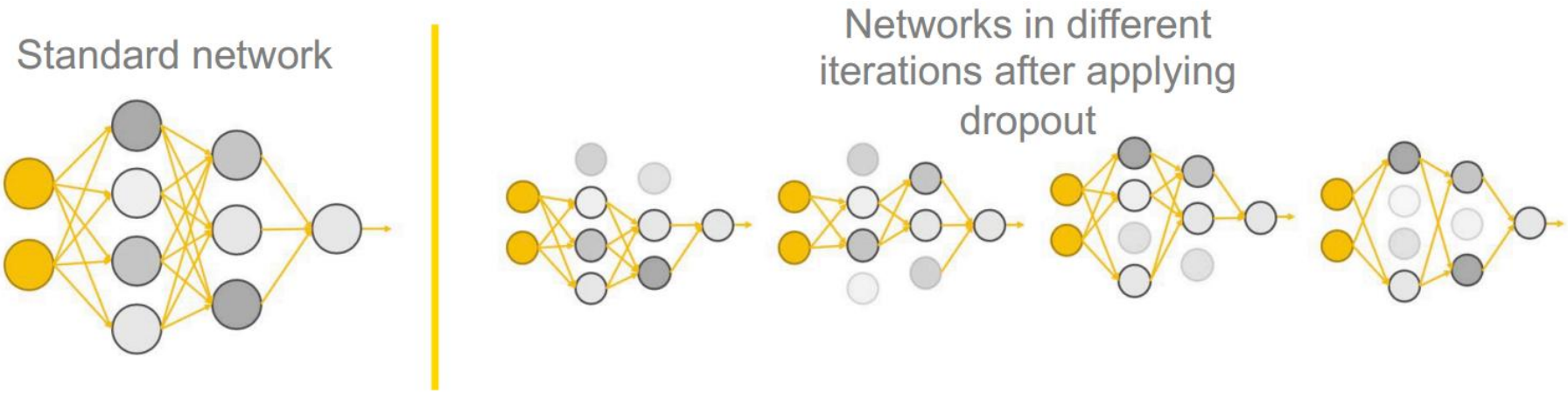
Avoiding Overfitting – Dropout

- **Randomly drop** neurons in input or hidden layer – introducing randomness.
- So that individual neurons do not rely on the input from a single neuron.
- Different neurons are dropped in each iteration.



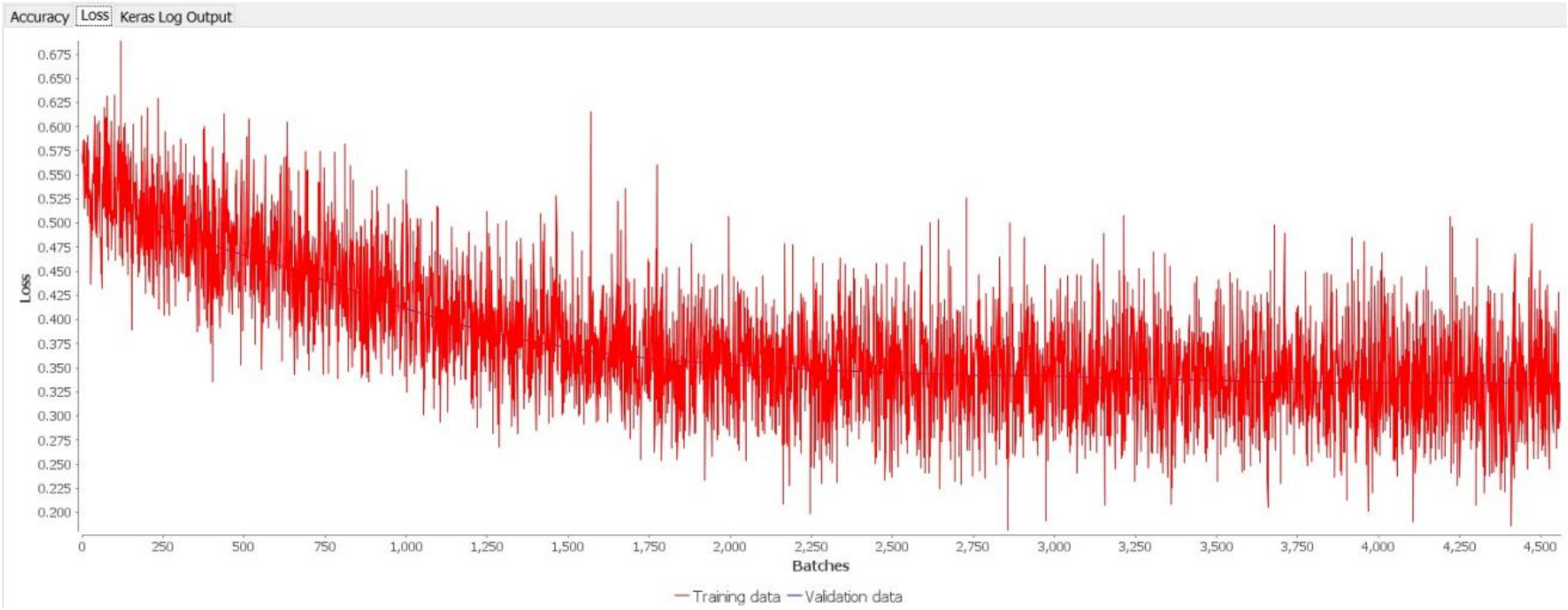
Avoiding Overfitting – Dropout

- **Dropout rate**
  - Proportion of nodes to be dropped
  - Typically, 20-50%
  - Too low → does not improve overfitting
  - Too high → network no longer able to predict



Avoiding Overfitting – Early Stopping

- Stops training the model when the loss function stops improving



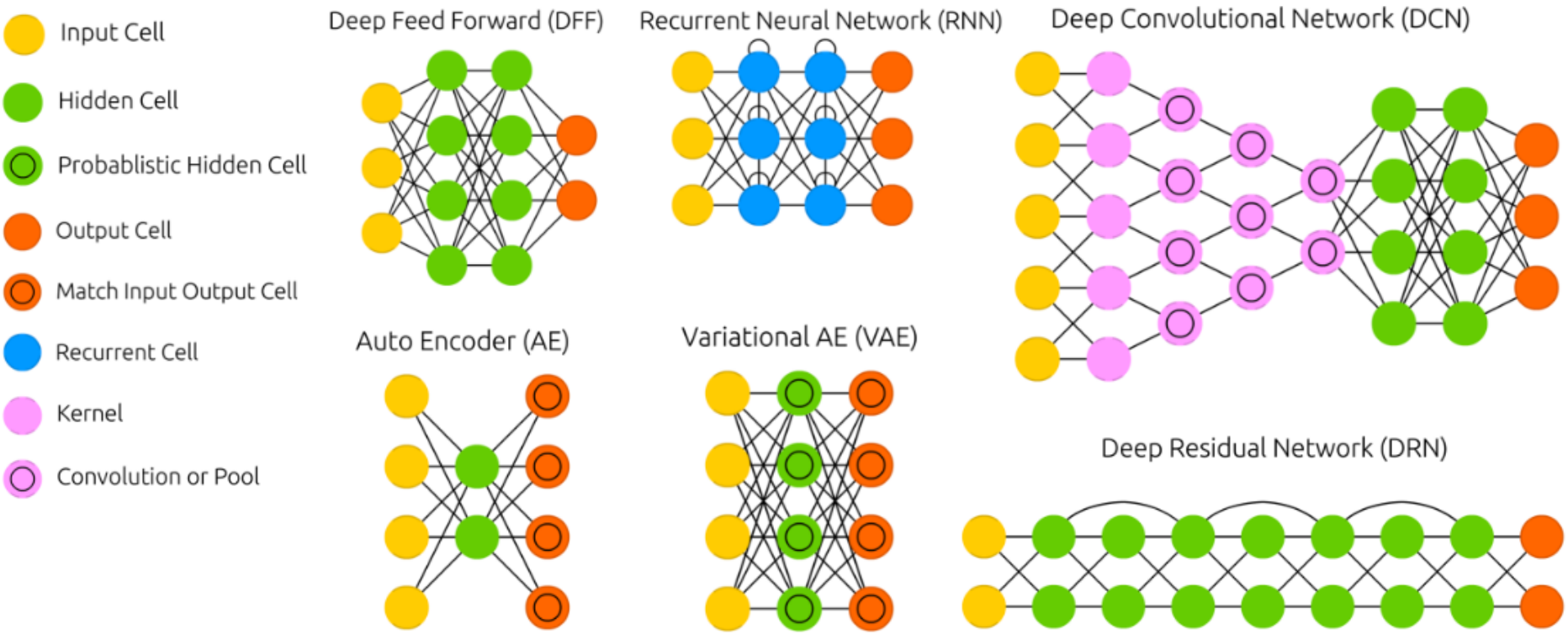
Best practices for training classifiers

- **Goal:** obtain a classifier with good generalization or performance on never-before-seen data.
  1. Learn parameters on the training set
  2. Tune hyperparameters (implementation choices) on the held-out validation set.
  3. Evaluate performance on the test set
     - *Crucial*: do not peek at the test set when iterating steps 1 and 2!

Neural Network Architectures

- **Standard Neural Networks**
  - Perceptron
  - Feed-Forward Networks
  - Residual neural network
- Advanced Architecture
  - Processing sequential data
    - Recurrent Neural Networks (RNNs)
    - The Long Short Term Memory Network (LSTM)
  - Convolutional Neural Networks (CNNs)
  - VAE
  - Autoencoder
  - Others

Neural Network Architectures



Convolutional Neural Networks (CNNs)

Conclusions

- Questions and Comments